LAB 7: Fuzzing for Input Validation Bugs

Name	Bandugula Sai Rithesh Reddy
SRN	PES1UG24AM064
Course	Software Engineering Lab
Assigned program	L6_L7_02 - Car Rental Rate (Lab 7/processor.py)
Tools	Python 3.14.0, Hypothesis 6.170.15, pytest 9.2.0

Objective

Use property-based fuzz testing with Hypothesis to find input-validation bugs in **processor.py** (license plate sanitising, trip mileage CSV parsing and rental invoice formatting), then harden the code so that no fuzzed input can crash it.

Step 1: Installing dependencies

pip install hypothesis and *pip install pytest*.

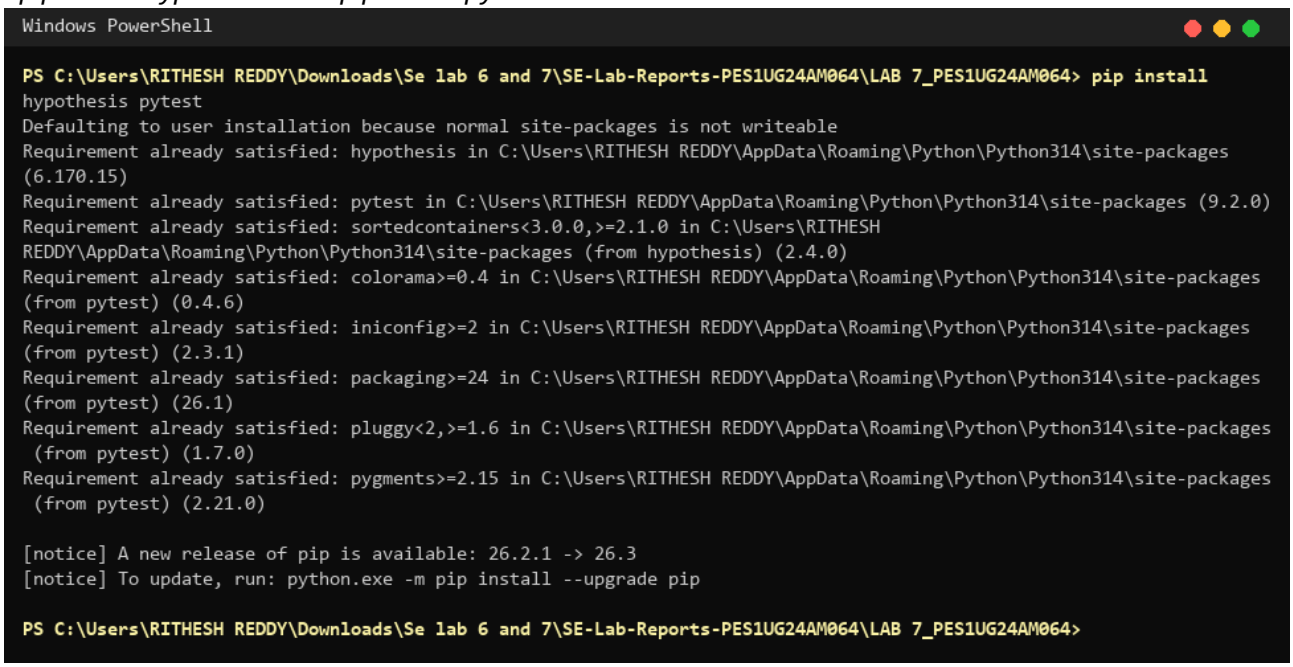


Figure 1 - hypothesis and pytest installed

Step 2: Running the buggy processor.py

Command: *python processor.py*. Run 1 uses well-formed input and works. Run 2 leaves an empty field in the mileage CSV (*100.5,,80.0*) and crashes with **ValueError**. Run 3 types the days as a word (*Ravi:three:SUV*) and crashes with **ValueError** inside *format_rental_invoice*.

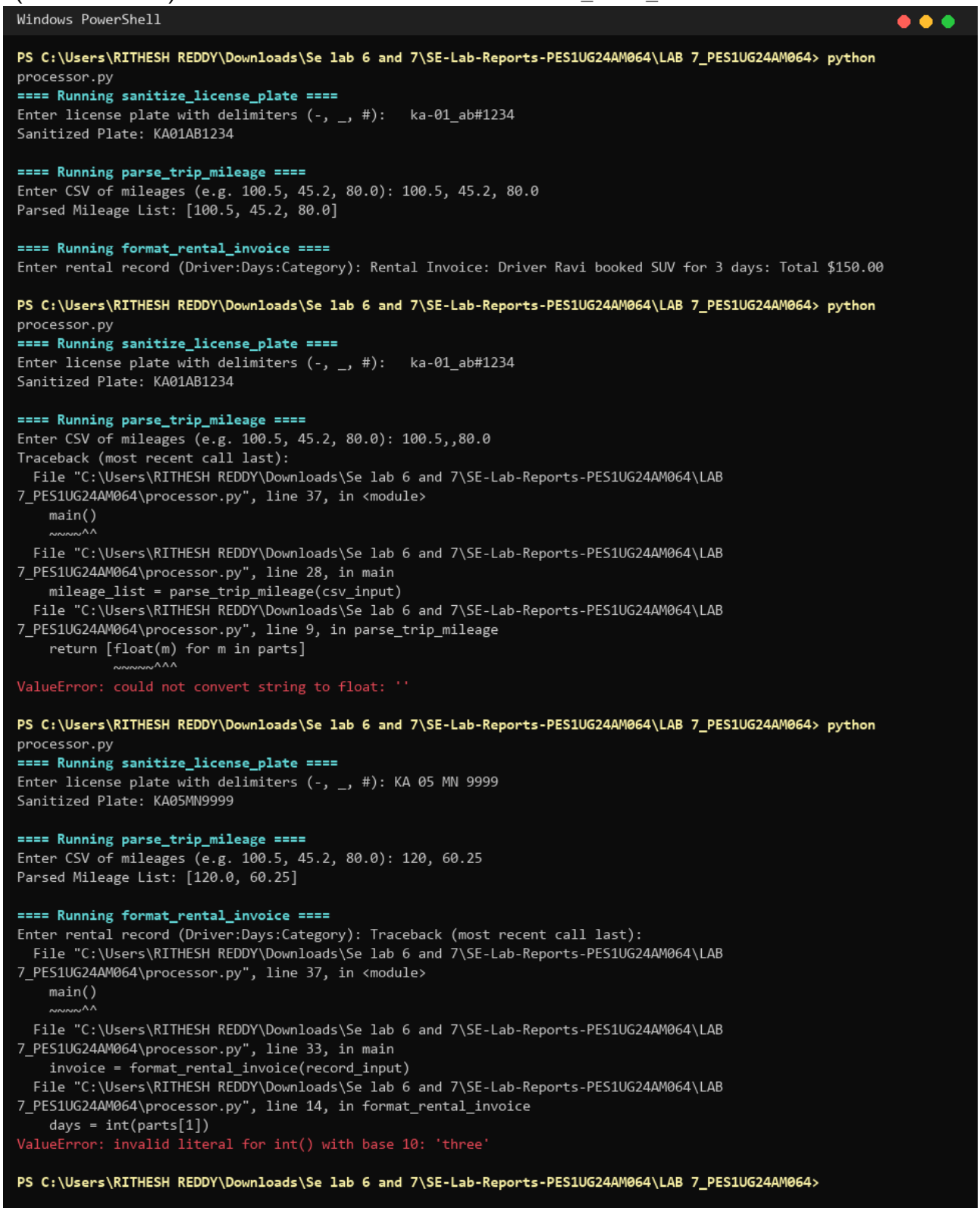


Figure 2 - buggy processor.py output (crashes on malformed input)

Step 3: Completing test_processor.py with Hypothesis

Each of the three starter tests was completed with try/except: the function is called with *st.text()* | *st.none()* and any exception fails the test with the input that caused it. After the call, the result is checked as well: a string with no delimiters for the plate, a list of finite non-negative floats for mileage, and a string for the invoice.

Random text rarely looks like a CSV of numbers or a *Driver:Days:Category* record, so two structured fuzzers were added. They build inputs from *st.floats(allow_nan=True, allow_infinity=True)* and *st.integers(-1000, 1000)*, which lets Hypothesis reach nan, inf, negative mileage and zero/negative rental days.

Test	Strategy	Property checked
test_sanitize_license_plate_no_crash	st.text() | st.none()	no exception; returns str; no '-', ' ', '_', '#' left
test_parse_trip_mileage_safe	st.text() | st.none()	no exception; list of finite floats >= 0
test_parse_trip_mileage_csv_like	comma-joined floats (nan/inf/negative) and short text	no exception; list of finite floats >= 0
test_format_rental_invoice_safe	st.text() | st.none()	no exception; returns str
test_format_rental_invoice_record_like	Driver:Days:Category from text, integers, categories	no exception; a billed invoice has days > 0 and total > 0

Step 4: Running test_processor.py against the buggy code

Command: *python -m pytest test_processor.py --tb=line* (--tb=line keeps the tracebacks short). All **5 tests fail**. Hypothesis shrank each failure to a minimal input:

Function	Minimal failing input	Failure
sanitize_license_plate	None	AttributeError: 'NoneType' object has no attribute 'strip'
parse_trip_mileage	None	AttributeError: 'NoneType' object has no attribute 'split'
parse_trip_mileage	'' (empty string / empty CSV field)	ValueError: could not convert string to float: ''
parse_trip_mileage	'INF', '-1.0' (from an earlier fuzzing run)	inf and negative mileage accepted without error
format_rental_invoice	None	AttributeError: 'NoneType' object has no attribute 'split'
format_rental_invoice	''	IndexError: list index out of range (fewer than 3 fields)
format_rental_invoice	':' and '::SUV'	ValueError: invalid literal for int() with base 10: ''
format_rental_invoice	':0:SUV' (and ':-1:SUV' in an earlier run)	zero/negative days billed: 'for 0 days: Total $0.00', 'Total $-50.00'



Figure 3.1 - pytest output on the buggy processor.py (part 1)

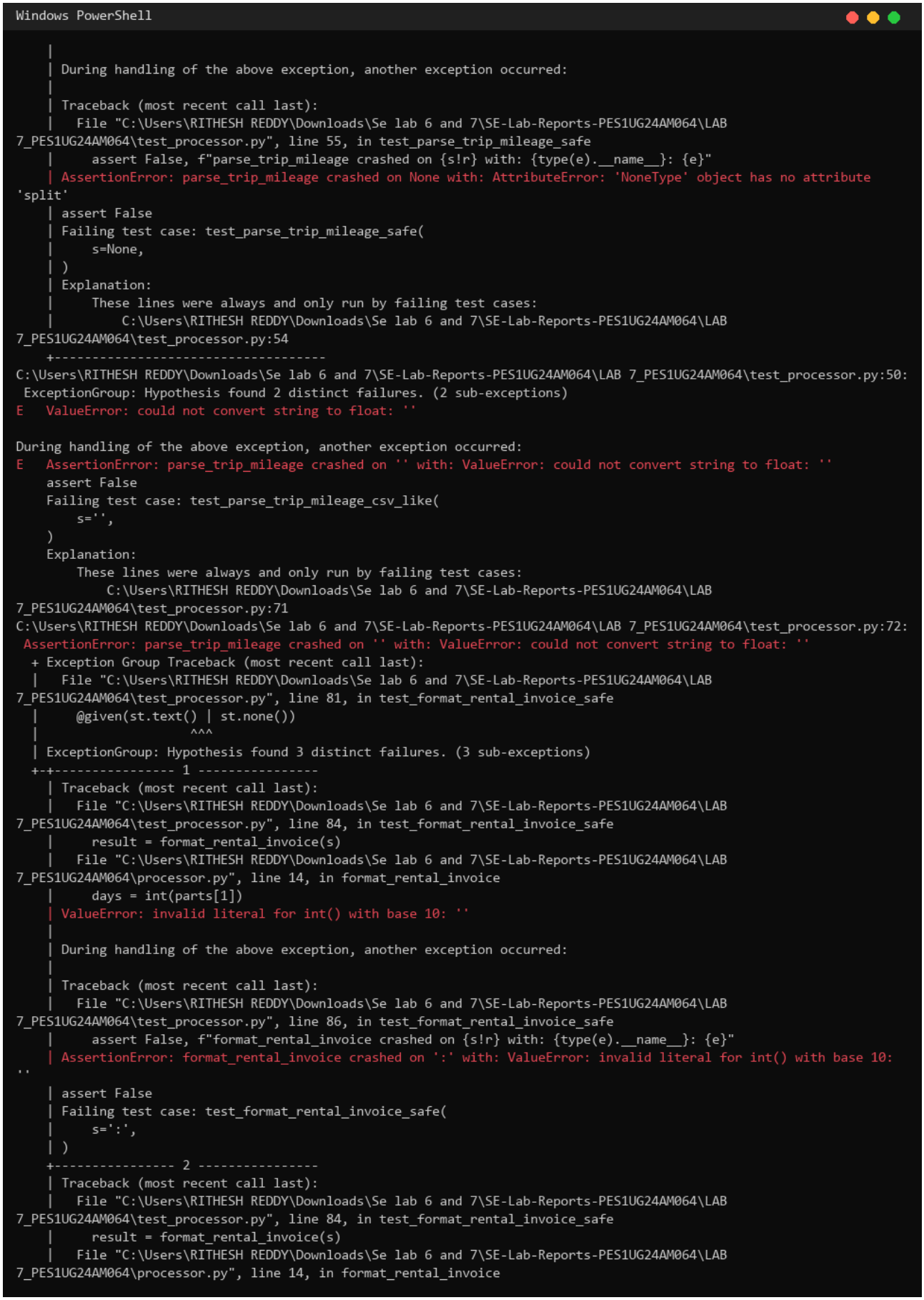


Figure 3.2 - pytest output on the buggy processor.py (part 2)



Figure 3.3 - pytest output on the buggy processor.py (part 3)

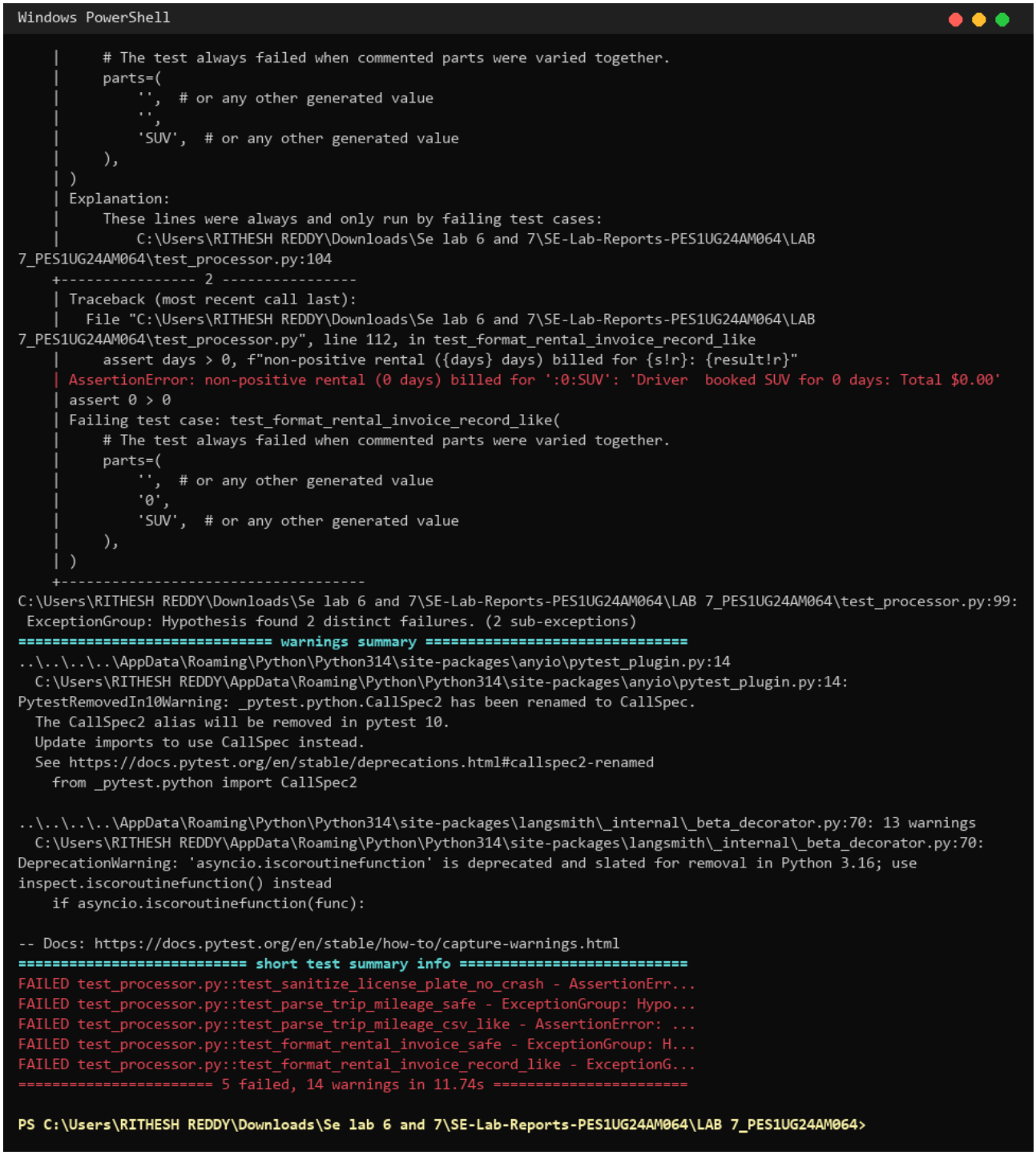


Figure 3.4 - pytest output on the buggy processor.py (part 4)

Step 5: Fixing processor.py

Function	Fix
sanitize_license_plate	Returns '' for non-string input (e.g. None). Also removes tabs/newlines inside the plate.
parse_trip_mileage	Returns [] for non-string input. Each field is stripped; empty fields, non-numeric text, nan/inf and negative values are skipped instead of raising or being accepted.
format_rental_invoice	Returns 'Invalid rental record: <reason>' for non-string input, a field count other than 3, a missing driver or category, non-integer days, or days outside 1..365. Days must be plain ASCII digits, so no int() exception is possible.
main()	Prints a note when mileage values are skipped and an '<empty>' message for an empty plate. Exits cleanly on end of input (EOFError/KeyboardInterrupt).

Command: *python processor.py*. The same inputs that crashed the buggy version now give clear results, and the valid input still gives the same output as before.

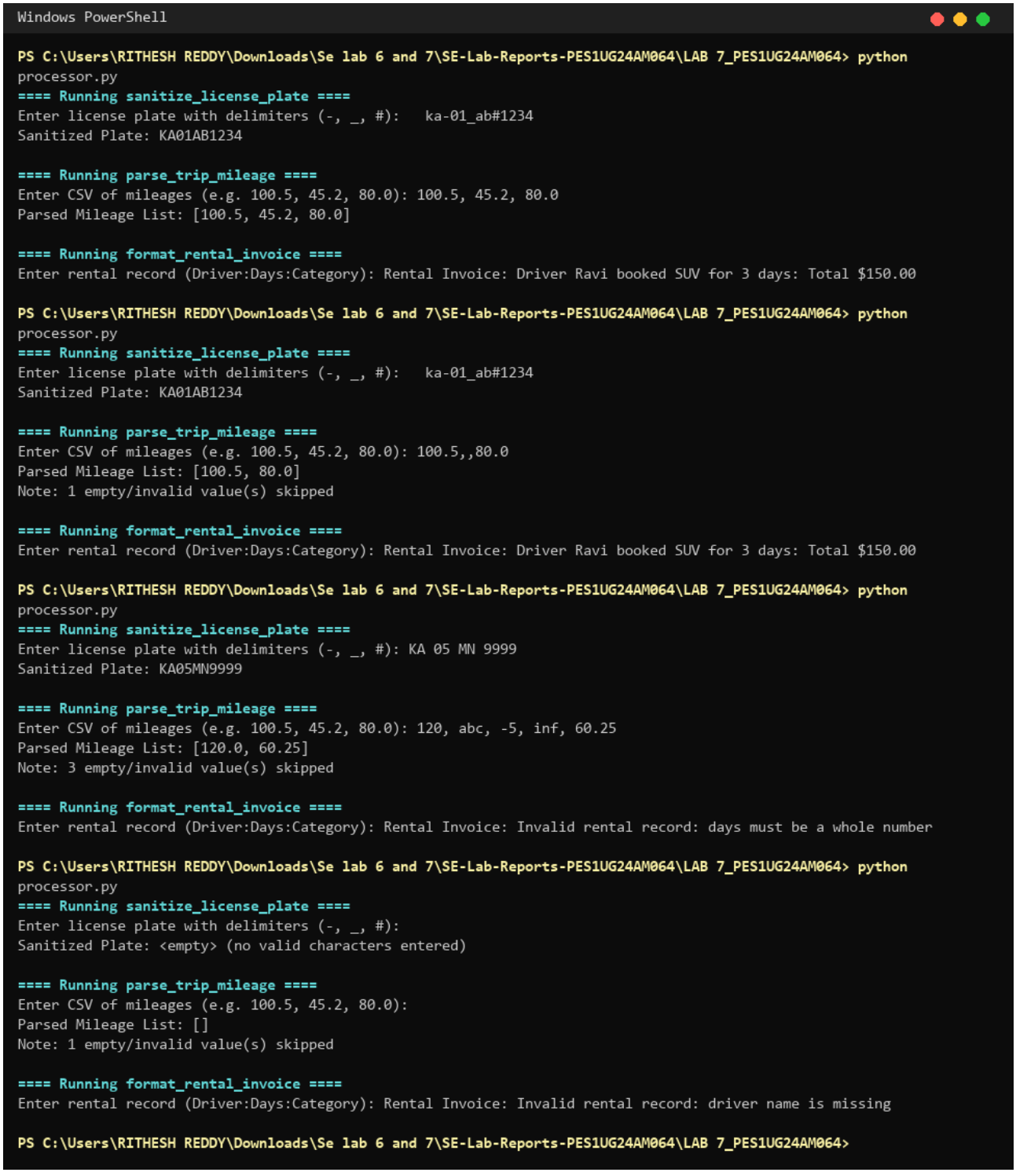


Figure 4 - fixed processor.py output

Step 6: Re-running test_processor.py against the fixed code

Command: *python -m pytest test_processor.py*. All **5 tests pass**, each with 500 generated examples. The run was repeated with 3 more random seeds and passed every time. The warnings come from unrelated pytest plugins (anyio, langsmith) installed on this machine, not from the lab code.

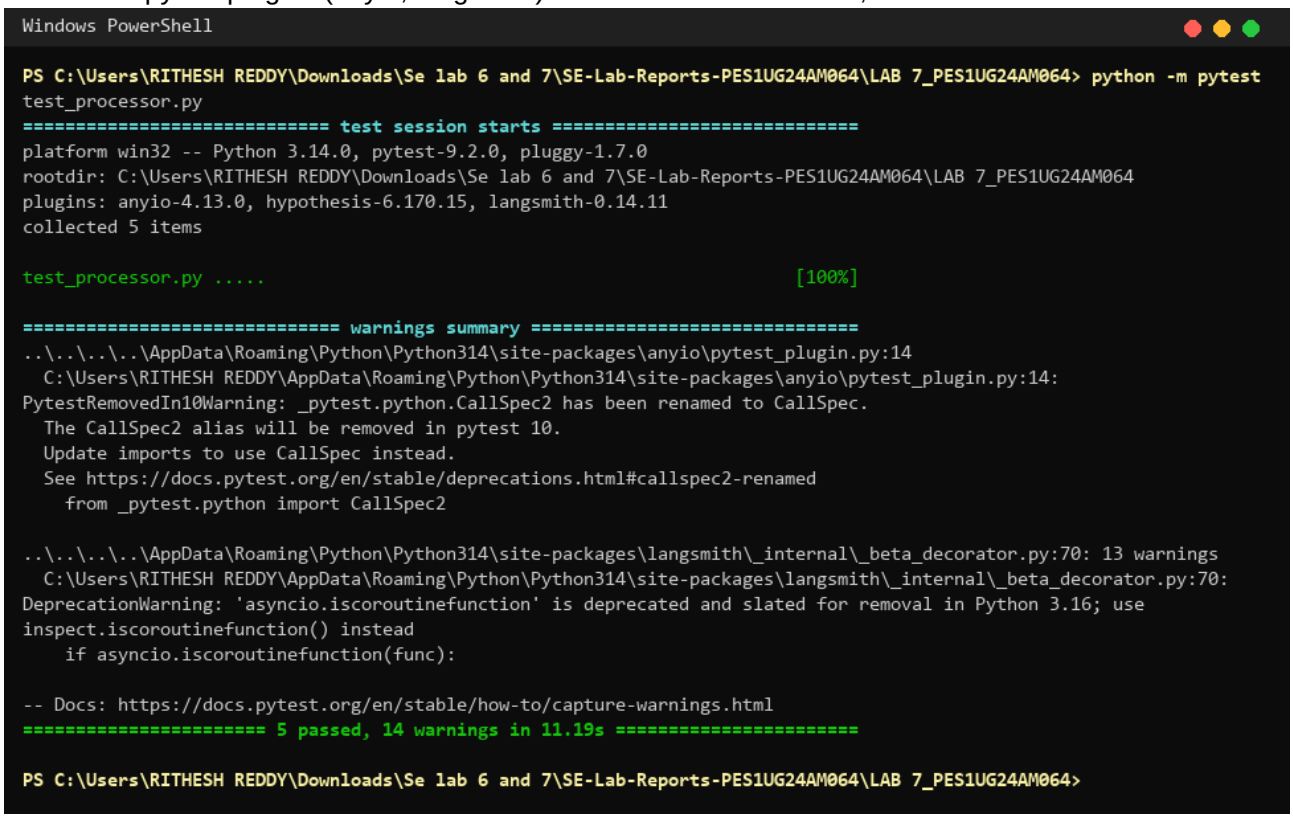


Figure 5 - all fuzz tests pass on the fixed processor.py

Conclusion

The starter tests only had to say what must always hold (no crash, a sane return value), and Hypothesis found and shrank the failing inputs itself: None, empty strings, missing fields, non-finite and negative numbers, and zero-day rentals. Most of these are easy to miss with hand-written unit tests. After defensive checks were added to every function, the same fuzzers find no failures.

Appendix A - Fixed processor.py

```
"""Car rental input processor - hardened version (Lab 7).

Every function validates its input and returns a safe value instead of
raising, so random / malformed input found by fuzzing can no longer crash it.
"""
import math

PLATE_DELIMITERS = ['-', ' ', '_', '#']
SUV_DAILY_RATE = 50.0
DEFAULT_DAILY_RATE = 30.0
MAX_RENTAL_DAYS = 365
INVALID_RECORD = "Invalid rental record"


def sanitize_license_plate(plate):
    """Remove delimiters/whitespace and upper-case the plate.

    Non-string input (e.g. None) returns an empty string.
    """
    if not isinstance(plate, str):
        return ""
    plate = plate.strip()
    for ch in PLATE_DELIMITERS:
        plate = plate.replace(ch, '')
    # Drop any remaining whitespace such as tabs or newlines inside the plate.
    plate = "".join(plate.split())
    return plate.upper()


def parse_trip_mileage(mileage_csv):
    """Parse a comma separated list of mileages.

    Empty fields, non-numeric text, nan/inf and negative values are skipped.
    Non-string input returns an empty list.
    """
    if not isinstance(mileage_csv, str):
        return []
    mileages = []
    for part in mileage_csv.split(','):
        part = part.strip()
        if not part:
            continue
        try:
            value = float(part)
        except ValueError:
            continue
        if not math.isfinite(value) or value < 0:
            continue
        mileages.append(value)
    return mileages


def format_rental_invoice(record):
    """Build an invoice line from a 'Driver:Days:Category' record.

    Malformed records return a message starting with 'Invalid rental record'
    instead of raising.
    """
    if not isinstance(record, str):
        return f"{INVALID_RECORD}: input must be text"

    parts = record.split(':')
    if len(parts) != 3:
        return f"{INVALID_RECORD}: expected Driver:Days:Category"

    driver = parts[0].strip()
    days_text = parts[1].strip()
    category = parts[2].strip()

    if not driver:
        return f"{INVALID_RECORD}: driver name is missing"
    if not category:
        return f"{INVALID_RECORD}: category is missing"
    if not (days_text.isascii() and days_text.isdigit()):
        return f"{INVALID_RECORD}: days must be a whole number"
```

```
    days = int(days_text)
    if days < 1 or days > MAX_RENTAL_DAYS:
        return f"{INVALID_RECORD}: days must be between 1 and {MAX_RENTAL_DAYS}"

    daily_rate = SUV_DAILY_RATE if category.lower() == "suv" else DEFAULT_DAILY_RATE
    total = days * daily_rate
    return f"Driver {driver} booked {category} for {days} days: Total ${total:.2f}"


def main():
    print("==== Running sanitize_license_plate ====")
    raw_plate = input("Enter license plate with delimiters (-, _, #): ")
    clean_plate = sanitize_license_plate(raw_plate)
    if clean_plate:
        print("Sanitized Plate:", clean_plate)
    else:
        print("Sanitized Plate: <empty> (no valid characters entered)")

    print("\n==== Running parse_trip_mileage ====")
    csv_input = input("Enter CSV of mileages (e.g. 100.5, 45.2, 80.0): ")
    mileage_list = parse_trip_mileage(csv_input)
    print("Parsed Mileage List:", mileage_list)
    skipped = len(csv_input.split(',')) - len(mileage_list)
    if skipped:
        print(f"Note: {skipped} empty/invalid value(s) skipped")

    print("\n==== Running format_rental_invoice ====")
    record_input = input("Enter rental record (Driver:Days:Category): ")
    invoice = format_rental_invoice(record_input)
    print("Rental Invoice:", invoice)


if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nInput ended - exiting.")
```

Appendix B - Updated test_processor.py

```
"""Lab 7 - Fuzz tests for processor.py using Hypothesis.

The first three tests are the ones from the starter file: they feed random
Unicode text (and None) into each function and check that it never crashes
and that the returned value has the expected type/shape.

The last two tests build "almost valid" inputs (CSV of numbers, Driver:Days:
Category records) so Hypothesis can also reach edge cases that plain random
text rarely hits, such as nan/inf/negative mileage or zero/negative days.
"""
import math
import re

from hypothesis import given, settings, strategies as st

from processor import sanitize_license_plate, parse_trip_mileage, format_rental_invoice

DELIMITERS = ['-', ' ', '_', '#']
INVOICE_TAIL = re.compile(r"for (-?\d+) days: Total \$(-?\d+\.\d+)$")


# ---------------------------------------------------------------------------
# 1. sanitize_license_plate
# ---------------------------------------------------------------------------
@settings(max_examples=500)
@given(st.text() | st.none())
def test_sanitize_license_plate_no_crash(s):
    try:
        result = sanitize_license_plate(s)
    except Exception as e:
        assert False, f"sanitize_license_plate crashed on {s!r} with: {type(e).__name__}: {e}"

    assert isinstance(result, str), f"expected str, got {type(result).__name__}"
    for ch in DELIMITERS:
        assert ch not in result, f"delimiter {ch!r} left in output {result!r}"


# ---------------------------------------------------------------------------
# 2. parse_trip_mileage
# ---------------------------------------------------------------------------
def check_mileage_list(s, result):
    assert isinstance(result, list), f"expected list, got {type(result).__name__}"
    for m in result:
        assert isinstance(m, float), f"non-float value {m!r} in result for {s!r}"
        assert math.isfinite(m), f"non-finite mileage {m!r} accepted from {s!r}"
        assert m >= 0, f"negative mileage {m!r} accepted from {s!r}"


@settings(max_examples=500)
@given(st.text() | st.none())
def test_parse_trip_mileage_safe(s):
    try:
        result = parse_trip_mileage(s)
    except Exception as e:
        assert False, f"parse_trip_mileage crashed on {s!r} with: {type(e).__name__}: {e}"

    check_mileage_list(s, result)


# CSV built from real floats (including nan, inf, negatives), random words and
# empty fields, joined with commas and optional spaces.
mileage_token = st.floats(allow_nan=True, allow_infinity=True).map(str) | st.text(max_size=5)
mileage_csv = st.lists(mileage_token, max_size=6).map(", ".join)


@settings(max_examples=500)
@given(mileage_csv)
def test_parse_trip_mileage_csv_like(s):
    try:
        result = parse_trip_mileage(s)
    except Exception as e:
        assert False, f"parse_trip_mileage crashed on {s!r} with: {type(e).__name__}: {e}"

    check_mileage_list(s, result)
# ---------------------------------------------------------------------------
```

```
# 3. format_rental_invoice
# ------------------------------------------------------------------------
@settings(max_examples=500)
@given(st.text() | st.none())
def test_format_rental_invoice_safe(s):
    try:
        result = format_rental_invoice(s)
    except Exception as e:
        assert False, f"format_rental_invoice crashed on {s!r} with: {type(e).__name__}: {e}"

    assert isinstance(result, str), f"expected str, got {type(result).__name__}"


# Records shaped like Driver:Days:Category with random parts, so the days
# field is often a real (possibly zero or negative) integer.
days_field = st.integers(min_value=-1000, max_value=1000).map(str) | st.text(max_size=4)
record = st.tuples(st.text(max_size=8), days_field,
                   st.sampled_from(["SUV", "suv", "Sedan", "", " "]) | st.text(max_size=6))


@settings(max_examples=500)
@given(record)
def test_format_rental_invoice_record_like(parts):
    s = ":".join(parts)
    try:
        result = format_rental_invoice(s)
    except Exception as e:
        assert False, f"format_rental_invoice crashed on {s!r} with: {type(e).__name__}: {e}"

    assert isinstance(result, str), f"expected str, got {type(result).__name__}"
    # A zero/negative rental must never produce a bill (or a negative total).
    billed = INVOICE_TAIL.search(result)
    if billed:
        days, total = int(billed.group(1)), float(billed.group(2))
        assert days > 0, f"non-positive rental ({days} days) billed for {s!r}: {result!r}"
        assert total > 0, f"non-positive total ${total} billed for {s!r}: {result!r}"
```